

ArrayList in Java: General Notes and Operations

This document provides a comprehensive overview of ArrayList operations in Java, including its declaration, key methods, and usage scenarios. It is designed as a quick reference guide for future use, highlighting practical examples and key points.

1. Declaration of ArrayList

ArrayList is a resizable array implementation provided by Java's Collection Framework. There are multiple ways to declare an ArrayList depending on the requirements:

- a) `ArrayList<String> temp = new ArrayList<>();`
 - Declares an ArrayList to store String elements only.
- b) `ArrayList<Integer> temp = new ArrayList<>();`
 - Declares an ArrayList to store Integer elements only.
- c) `ArrayList temp = new ArrayList<>();`
 - Declares a non-generic ArrayList that can store elements of any type.
- d) `List temp = new ArrayList<>();`
 - Declares the ArrayList with the List interface type, enabling polymorphism.

2. Common Methods and Their Usage

a) Adding Elements

The `add()` method is used to add elements to the ArrayList. It can append elements at the end or insert them at a specific index.

Example:

- `temp.add(50);` // Adds an Integer
- `temp.add("Hello");` // Adds a String
- `temp.add(2, "Python");` // Inserts 'Python' at index 2

b) Removing Elements

The `remove()` method removes elements either by their value or by their index position.

Example:

- `temp.remove("Hello");` // Removes the element 'Hello'
- `temp.remove(2);` // Removes the element at index 2

c) Retrieving Elements

The `get()` method retrieves the element at a specified index position.

Example:

- `String element = temp.get(2);` // Retrieves the element at index 2

d) Replacing Elements

The `set()` method replaces the element at a specified index with a new value.

Example:

- `temp.set(2, "C++");` // Replaces the element at index 2 with 'C++'

e) Checking Presence of an Element

The `contains()` method checks if a specific element exists in the `ArrayList`.

Example:

- `temp.contains("C++");` // Returns true if 'C++' is present

f) Size and Emptiness Check

- `size()`: Returns the number of elements in the `ArrayList`.

- `isEmpty()`: Returns true if the `ArrayList` is empty, otherwise false.

3. Iterating Through Elements

There are multiple ways to read elements from an `ArrayList`:

a) Using a for loop:

- Example: `for (int i = 0; i < temp.size(); i++) { System.out.println(temp.get(i)); }`

b) Using a for-each loop:

- Example: `for (Object element : temp) { System.out.println(element); }`

c) Using an Iterator:

- Example: `Iterator it = temp.iterator(); while (it.hasNext()) { System.out.println(it.next()); }`

4. Conclusion

`ArrayList` is a versatile and commonly used collection in Java. It allows dynamic resizing and supports various operations for efficient manipulation of elements. This guide outlines essential operations and serves as a handy reference for working with `ArrayLists`.